

EMBER

Estrategias para la reunion con los mentores del hackathon

Situacion de partida

El proyecto ya dispone de un baseline CPU funcional, reproducible y medible. La reunion debe servir para elegir una unica primera aceleracion y acordar como demostrarla con rigor.

Mensaje principal

Tenemos un simulador C++17 con SoA, doble bufer, vecindario de Moore, RNG determinista por hash, CLI, exportacion y tests. El siguiente paso no es anadir funcionalidades al azar: es conservar esta implementacion como oracle de correccion y medir una aceleracion comparable.

Objetivo concreto de la reunion

- Fijar la metrica principal: speedup respecto al baseline, throughput, escalabilidad o eficiencia.
- Elegir la primera ruta: OpenMP por escenarios, AVX2 dentro del kernel o CUDA.
- Acordar hardware, profiler y protocolo de benchmark disponibles en MareNostrum.
- Definir que significa reproducibilidad entre el backend CPU y una futura GPU.

Fecha de preparacion: 15 de septiembre de 2026

1. Estado actual que conviene presentar

Area	Hecho	Lectura para la reunion
Nucleo	C++17, SoA, doble bufer, Moore de 8 vecinos, viento, humedad, vegetacion, pendiente y combustible.	La estructura es local por celda y portable a GPU.
Reproducibilidad	Hash de 64 bits derivado de semilla, escenario, paso y celda.	No depende del orden de recorrido; buena base para OpenMP/CUDA.
Validacion	CTest pasa; ember_tests contiene 10 grupos de casos.	La referencia de correccion existe antes de optimizar.
Interfaz	CLI, estadisticas, CSV y PPM.	Permite demo y benchmark sin GUI.
Documentacion	README, modelo simplificado, notas CUDA y Doxygen.	La historia tecnica se puede explicar a terceros.

Evidencia que ya podeis enseñar

La validacion actual pasa en el build principal y en los builds experimentales avx2/scalar. El benchmark CSV contiene 80 filas: 40 de Windows y 40 de un entorno Linux. El throughput combinado varia aproximadamente entre 17 y 69 millones de cell updates/s, por lo que esos datos deben normalizarse antes de usarlos para afirmar un speedup.

Rama experimental

Existe una rama Git llamada avx (commit 3131237) con una ruta AVX2 opcional, deteccion de soporte en runtime y fallback escalar. La rama main sigue siendo la referencia. Presentad AVX2 como experimento reproducible pendiente de integracion, no como una funcionalidad ya consolidada en main.

2. Preguntas de alto valor para los mentores

Tema	Pregunta exacta	Postura recomendada
Criterio de exito	Que valorara mas el jurado: speedup, throughput, escalabilidad, energia o modelo?	Proponer speedup reproducible frente al baseline escalar.
Primera optimizacion	Priorizamos OpenMP por escenarios, AVX2 o CUDA?	Perfilar primero; despues OpenMP por escenarios y luego kernel/AVX2 o CUDA.
Benchmark	Usamos pasos fijos, trabajo real o las dos medidas?	Reportar siempre tiempo, cell_updates, throughput y motivo de terminacion.
Profiling	Que herramientas y particiones de MareNostrum tendremos?	Elegir el profiler antes de tocar el kernel.
Reproducibilidad	Esperais igualdad bit a bit entre CPU y GPU?	Igualdad estricta dentro de cada backend; equivalencia funcional entre backends.
Modelo	Basta el modelo educativo o conviene sensibilidad/correlacion espacial?	Mantener el modelo estable y anadir sensibilidad solo si mejora la evaluacion.

Frase util para encauzar la conversacion

Propuesta de decision

Queremos salir de la reunion con una metrica objetivo, una herramienta de perfilado, un hardware de referencia y una primera optimizacion acotada que podamos validar contra el baseline.

No conviene prometer

- Prediccion operativa de incendios: el modelo es educativo.
- Comparacion directa entre los benchmarks actuales: mezclan maquinas y compiladores.
- Speedup AVX2 definitivo: la rama aun no esta integrada en main.
- CUDA inmediata: primero hay que congelar la referencia y el protocolo.

3. Lectura tecnica: donde esta el trabajo interesante

Hotspot probable

El coste dominante esta en el bucle de actualizacion temporal: cada celda visita hasta ocho vecinos, comprueba bordes, lee varios arrays SoA, aplica modificadores y calcula un hash para la decision probabilistica. El codigo esta en src/simulation.cpp, alrededor de los bucles de filas, columnas y vecinos.

- Perfilar si el limite real es ancho de banda, calculo, ramas o el hash determinista.
- Medir el coste de las comprobaciones de borde y valorar una ruta interior separada.
- Evitar optimizaciones que cambien la semantica sin compararlas con la referencia.
- Mantener la E/S fuera del tiempo de simulacion y registrar trabajo real ejecutado.

Rutas de aceleracion

Ruta	Ventaja	Riesgo / condicion
OpenMP por escenarios	Cambio pequeno; escenarios independientes; buen primer control de escalabilidad.	Necesita preservar el orden de resultados y medir saturacion de memoria.
AVX2	Ataca el kernel de celdas y sirve como aprendizaje SIMD CPU.	Debe integrarse en CMake/main y compararse en la misma maquina.
CUDA	Una hebra por celda, arrays residentes y doble bufer encajan bien.	Reducciones, divergencia, transferencia y diferencias flotantes.

Puntos CUDA que ya estan preparados

- SoA y buffers current/next permiten intercambio de punteros en dispositivo.
- El hash sin estado evita RNG compartido y dependencias del orden.
- El kernel por celda puede leer el vecindario local y escribir un destino independiente.
- Las reducciones necesarias son: celdas Burning, celdas quemadas y deteccion de extincion.

4. Plan de accion y demo

Etapa	Resultado	Criterio de salida
1. Congelar baseline	Tag/commit, tests y benchmark en una sola maquina.	Misma semilla y workload; resultados guardados con metadata.
2. Perfilar	Hotspots y contadores de memoria/calculo.	Decision basada en datos, no en intuicion.
3. OpenMP	Escenarios paralelos con 1/2/4/8 hilos.	Speedup y resultados no temporales equivalentes.
4. SIMD o CUDA	Primer kernel optimizado.	Comparacion contra baseline y explicacion de trade-offs.
5. Historia final	Tabla de speedup, metodo y limites.	Demo reproducible y candidatura defendible.

Demo Windows de dos minutos

```
build\Release\ember.exe --width 256 --height 256 --steps 100 --scenarios 4 --seed 42
--wind-direction 45 --wind-strength 0.4 --base-spread 0.25 --output results\mentor-demo
--export both
```

- Enseñar la salida de throughput y los escenarios extinguidos/max_steps.
- Abrir summary.csv y una imagen scenario_000000_final.ppm.
- Repetir el comando con la misma semilla para mostrar reproducibilidad.
- Cerrar con una pregunta concreta: que optimizacion y profiler priorizamos?

Checklist para llevar

Portatil con la build Release, copia del README, benchmark CSV, grafico simple de throughput por workload, y una lista de cuatro preguntas de decision. No hace falta llevar CUDA funcionando para que la reunion sea productiva.

Archivos de referencia

README.md | src/simulation.cpp | include/ember/grid.hpp | include/ember/random.hpp | docs/model.md | docs/cuda-readiness.md | benchmarks/benchmark_results.csv